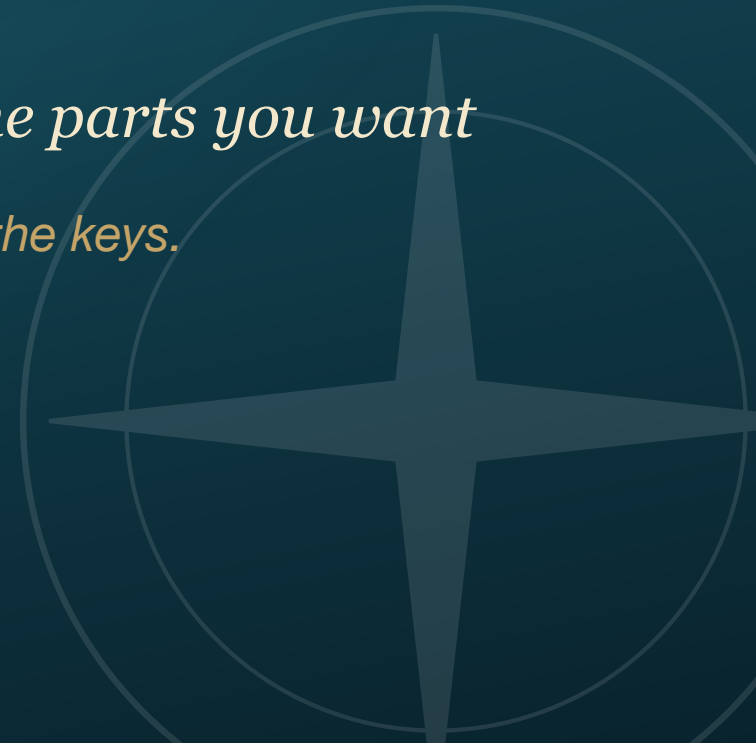


DAY 4 • LECTURE & DEMO • ~25 MINUTES

Make It Your Own

Forking the Island: take the whole seminar — or just the parts you want

Three days you sailed this ship. This morning, here are the keys.



The real takeaway is a git repo

- ♦ Everything you have touched this week — the slides, the lab missions, the cluster, the AI mate — lives in **one repository**.
- ♦ It is **public**. You can clone it today and run this exact seminar at your own college.
- ♦ It is **modular**. Take the whole thing, or lift one piece into a course you already teach.
- ♦ This session is the map. Four questions:
 - **Where is everything?** (the layout)
 - **What do I change?** (`lab.env`)
 - **How do I run it?** (the `justfile`)
 - **How do I make it *mine*?** (docs, slides, the AI persona)

PART I

One repo, both halves

curriculum	infra
docs/	k8s/
slides/	justfile

One hull carries both the cargo and the crew.

The map: what lives where

Path	What it is
<code>justfile</code>	Every operational command. <code>just</code> with no args lists them all.
<code>lab.env.example</code>	The config template — copy to <code>lab.env</code> , fill in, never commit.
<code>k8s/</code>	Kubernetes manifests + Helm values for every service.
<code>scripts/</code>	Cluster bootstrap, student provisioning, client setup.
<code>docs/</code>	The curriculum — a MkDocs site (missions, one-pagers, quizzes).
<code>slides/</code>	These decks — Marp markdown → HTML + PDF.
<code>terraform/</code>	Optional OpenTofu stack to provision a cloud VM.
<code>certs/</code>	Generated lab TLS (private keys gitignored).

PART II

The one knob that matters

lab.env

Set the heading once; every recipe sails by it.



`lab.env` — your institution in one file

```
cp lab.env.example lab.env      # gitignored –  
holds YOUR values
```

- ♦ `SERVER_IP` **blank** → **LOCAL mode** — everything runs on a k3d cluster on your laptop.
- ♦ `SERVER_IP` **set** → **REMOTE mode** — the same recipes run over SSH against your server.
- ♦ The other fields you actually edit before a real class:
 - `LAB_DOMAIN` — the domain behind every service URL.
 - `AI_API_KEY`, `*_ADMIN_PASSWORD` — **change these from the defaults.**
 - `ORG_*` — branding baked into your TLS certificate.
- ♦ One file selects laptop-demo vs. full classroom. Nothing else changes.

PART III

The control panel

the `justfile`

A complete deployment is a short, ordered list of `just` commands.

`just` — the whole lab as commands

```
just          # list every recipe, grouped and described
just init     # validate your config + tooling
just bootstrap-k3d # stand up a local cluster
just deploy-core # gateway, AI engine, docs, polls...
just deploy-gitea # ...then each tool, in order
just provision # create every student from a roster CSV
just serve-docs # preview the curriculum at :8000
just slides   # render every deck to HTML + PDF
```

- ♦ You never memorize `kubectl`/`helm` incantations — the recipes are the runbook.
- ♦ Read any recipe to see exactly what it does. The `justfile` **is** the documentation.

The toolbox — and every tool is swappable

Service	Role	Swap it for
Rancher	Cluster UI / kubeconfig	any K8s dashboard
Gitea	Git server	GitHub / GitLab
Harbor	Image registry	any OCI registry
ArgoCD	GitOps delivery	Flux
Dex	Single sign-on	your campus IdP
Ollama + LiteLLM	The AI mate	any OpenAI-compatible API
MkDocs	Course site	any static-site generator
Quizler	In-class polls	your LMS quiz tool

PART IV

Making it yours docs · slides · the AI mate

The parts you will actually rewrite.

The curriculum is markdown: docs/

- ♦ The course site is **MkDocs** — every mission, one-pager, instructor guide, and quiz is a markdown file under `docs/`.
- ♦ Preview your edits live:

```
just serve-docs      # http://localhost:8000,  
reloads as you save
```

- ♦ Quizzes are plain `*.quizler` files loaded into the in-class poll app.
- ♦ Edit the markdown, commit, and — on a GitOps-wired cluster — the live site updates itself.

The slides are markdown too: Marp

```
---  
marp: true  
theme: nautical  
---  
  
# A slide is a heading  
  
- A bullet is a bullet  
---  
  
# The next slide starts after a triple-dash
```

- ♦ These decks are **plain text** — diffable, reviewable, no binary `.pptx`.
- ♦ The look comes from one shared theme: `slides/themes/nautical.css`.
- ♦ Render the whole set — HTML + PDF + the gallery page — with one recipe:

just slides

The teaching mate is a file: `AGENTS.md`

- ♦ The "Socratic Boatswain" is not a model setting — it is a **markdown persona file**.
- ♦ Students invoke it with `aichat -r boatswain`; the role is a symlink to `AGENTS.md`.
- ♦ **Rewrite the file → rewrite the TA**. You saw the proof an hour ago:
 - 3 days: a Socratic mate that *refuses to hand over answers*.
 - Mid-incident: overwrite it into an **Incident Commander** that gives the exact fix.
- ♦ Author your own personas — a strict syllabus enforcer, a code reviewer, a debate partner.

PART V

Take all of it — or just a plank

You do not have to adopt the whole ship.



Reuse in parts

- ♦ **Just the slides** — fork `slides/`, keep the Marp + theme pipeline, write your own decks.
- ♦ **Just the AI persona** — drop the `AGENTS.md` + `aichat` pattern into any course.
- ♦ **Just the capstone** — lift the Chaos Mesh "Pirate Strikes" game as a self-grading exam.
- ♦ **Just the infra** — run the `justfile` + `k8s/` stack as a sandbox cluster, bring your own labs.
- ♦ **The whole seminar** — fork, set `lab.env`, `just` your way to a running classroom.
- ♦ Every piece stands alone because every piece is just files in a folder.

PART VI

Let the agent help you remix it

You don't have to read the whole ship's manifest yourself.

The repo reads itself

- ♦ This repo is **agent-legible** by design:
 - the curriculum is **markdown**,
 - the operations are a self-documenting **justfile**,
 - the AI persona is a plain **AGENTS.md**.
- ♦ So point an agent *at the repo* and let it do the mapping:
 - open it in an agentic IDE — **Antigravity**, Claude Code, Cursor —
 - or paste the **public GitHub URL** into Claude or Gemini.
- ♦ Then ask: "*which parts of this fit the class I want to teach?*"

A prompt you can steal

```
Read this repo (README.md, docs/make-it-your-own.md, the justfile,  
docs/missions/). It's a forkable DevOps seminar. I want to teach  
<FORMAT> on <TOPIC> for <AUDIENCE>.
```

```
1) What can I reuse as-is?
```

```
2) What should I cut?
```

```
3) What's missing, and where would it slot in?
```

```
Give me a session-by-session outline citing specific repo files.
```

- ♦ Example: *"I want to teach a DevOps class focused on **GitOps** — what could I use from this repo?"*
- ♦ One prompt covers the whole room — each of you fills in your own format, topic, audience.
- ♦ Treat the answer as a **first draft to react to**, not gospel. You own the curriculum.

PART VI • DEMO

Live: ask Antigravity

```
> "Teach me GitOps from this repo..."  
|  
+--> [ reading README.md ]  
+--> [ reading justfile ]  
+--> [ reading docs/missions/day-03/ ]
```

Watch it read the repo and build the outline in real time.

Demo — what you're watching

- ♦ The repo is open in **Antigravity**; the agent runs in the **Agent Manager**.
- ♦ I paste the prompt above with **GitOps** as the topic.
- ♦ Watch it:
 - **read** the README, the `justfile`, and the Day 3 GitOps labs;
 - **cite the actual files** it would reuse — the Helm lab, the ArgoCD lab, the Gitea Actions demo;
 - **draft a session outline** mapped to those files.
- ♦ The takeaway: this is how you'll plan your *own* fork — in minutes, not a weekend.

Fork it — don't just clone it

- ♦ A **clone** is just a local copy — there's nowhere of *yours* on GitHub, and no clean way to get updates.
- ♦ A **fork** is *your* copy of the repo under your account or org:
 - your customizations live there,
 - and you can **pull updates** from the original as the seminar improves.
- ♦ One-time setup, then update anytime:

```
git remote add upstream \  
  
https://github.com/ColumbusStateWorkforceInnovation/NITIC-seminar.git  
git fetch upstream && git merge upstream/main    #  
pull in the latest
```

The fork workflow, end to end

```
# 1. Fork on GitHub, then clone YOUR fork
git clone https://github.com/<you>/NITIC-seminar.git

# 1b. Point "upstream" at the original (once) – your update channel
git remote add upstream https://github.com/ColumbusStateWorkforceInnovation/NITIC-seminar.git

# 2. Configure your institution
cp lab.env.example lab.env          # edit domain, passwords, AI key

# 3. Add your roster (both files, same usernames)
cp scripts/students.csv.example scripts/students.csv
cp k8s/core-tools/dex.yaml.example k8s/core-tools/dex.yaml

# 4. Validate, deploy, provision
just init && just bootstrap-k3d && just deploy-core
just provision
```

- ♦ Full walkthrough lives in the repo: `docs/make-it-your-own.md` and the README.

Instructor Superpower

THE REPO IS THE PLATFORM

- ♦ **Version the curriculum and the cluster together** — one commit moves both.
- ♦ **Distribute the lab as a URL, not an install disk** — students `git clone` and run one script.
- ♦ **Run a local, file-based AI TA you fully control** — and re-tune it per lesson.
- ♦ You no longer wait for IT to bless a tool. You fork, you configure, you teach. The environments you can offer your students just expanded to *everything in this repo*.

It's yours now.

~~~~^~~~~~^~~~~~^~~~~~^~~~~~^~~~~~^~~~~~  
~~~~~^~~~~~^~~~~~^~~~~~^~~~~~^~~~~~^~~~~~  
~^~~~~~^~~~~~^~~~~~^~~~~~^~~~~~^~~~~~^~~~~~

Fork it tonight. And speaking of taking what isn't nailed down — the Pirate is almost aboard.

